

Deep Metric Learning for Image Retrieval

Muhammad Usman Muneer (BSCS22008)

March 28, 2026

1 Model Architecture

1.1 Network Design

The embedding network uses ResNet-50 as the feature extractor:

Architecture:

Input (224×224×3)
↓
ResNet-50 Backbone (ImageNet pretrained)
↓
Global Average Pool → 2048-d Features
↓
FC Layer (2048 → 128)
↓
L2 Normalization (Unit Sphere)
↓
Output: 128-d Embedding

Design Choices:

- **Backbone:** ResNet-50 with ImageNet pretraining for robust feature extraction
- **Feature Dimension:** 2048-d from final residual layer
- **Embedding Space:** 128-d projection layer reduces dimensionality while preserving discriminative information
- **Normalization:** L2 normalization constrains embeddings to unit sphere for Euclidean distance computation
- **Distance Metric:** Euclidean distance in normalized embedding space

Implementation: The ResNet-50 backbone loads pretrained ImageNet weights. The final fully connected layer is replaced with an Identity layer to extract 2048-d features. A learnable FC layer projects features to 128 dimensions. All embeddings are L2-normalized along the feature dimension to ensure unit norm, enabling Euclidean distance computation in normalized space.

2 Dataset Sampling Strategy

2.1 Data Split

Caltech-101 dataset (101 classes, 8,144 images) split as:

- **Training:** 70% (5,701 images)
- **Validation:** 15% (1,221 images)
- **Testing:** 15% (1,373 images)

2.2 Pair Sampling for Contrastive Learning

A PyTorch Dataset returns labeled pairs (x_1, x_2, y) where:

- $y = 1$: **Positive pair** — both images from the same class
- $y = 0$: **Negative pair** — images from different classes

Sampling Strategy: With probability $p = 0.5$:

- Sample positive pair: randomly select another image from anchor's class
- Sample negative pair: randomly select image from different class

This balanced sampling ensures the model learns both similarity and dissimilarity equally.

Implementation: The ContrastiveDataset maintains a dictionary mapping each class label to indices of images in that class. For each sample, at iteration a coin flip (probability $p = 0.5$) determines whether to sample a positive or negative pair. Positive sampling randomly selects another image from the anchor's class. Negative sampling randomly picks a different class, then randomly selects an image from that class. Both images are loaded, transformed (resized, augmented), and returned with the binary label (1 or 0).

2.3 Triplet Sampling

A PyTorch Dataset returns triplets (a, p, n) :

- **Anchor (a):** Reference image
- **Positive (p):** Different image from same class as anchor
- **Negative (n):** Image from different class

Constraint: Positive must be a different image than anchor to avoid trivial solutions.

Implementation: The TripletDataset maintains class-to-index mappings. For anchor at index i , positive sampling randomly selects from the same class but ensures the selected index differs from anchor (unless only one image exists in that class). Negative sampling picks a random different class and randomly selects an image from that class. All three images are loaded, transformed, and returned as a tuple $(anchor, positive, negative)$.

3 Loss Functions

3.1 Contrastive Loss

For a pair (x_1, x_2) with binary label $y \in \{0, 1\}$, compute Euclidean distance:

$$D = \|f(x_1) - f(x_2)\|_2$$

The contrastive loss is:

$$\mathcal{L}_c = y \cdot D^2 + (1 - y) \cdot \max(0, m - D)^2$$

where:

- $y = 1$: Positive pairs should have small distance
- $y = 0$: Negative pairs should exceed margin $m = 1.0$
- Margin m acts as a threshold for acceptable negative pair distances

Implementation: Pairwise Euclidean distances are computed between all pairs in the batch using PyTorch's `pairwise_distance`. For positive pairs ($y = 1$), the loss penalizes distance directly as D^2 . For negative pairs ($y = 0$), the loss applies margin constraint: distances below the margin incur quadratic penalty, while distances exceeding the margin incur zero loss. Batch mean aggregates the loss across all pairs.

3.2 Triplet Loss

For triplet (a, p, n) , the loss enforces:

$$D(a, p) + m < D(a, n)$$

Triplet loss is:

$$\mathcal{L}_t = \max(0, D(a, p) - D(a, n) + m)$$

where:

- $D(a, p)$: Distance from anchor to positive
- $D(a, n)$: Distance from anchor to negative
- $m = 0.2$: Margin ensuring minimum separation

Implementation: Pairwise distances are computed separately: $D(a, p)$ between anchors and positives, and $D(a, n)$ between anchors and negatives. The triplet loss computes the margin-based hinge loss: $\max(0, D(a, p) - D(a, n) + m)$. The clamping operation ensures zero loss when negative is sufficiently far from anchor. Batch mean aggregates across all triplets.

3.3 Batch-Hard Triplet Loss (Hard Negative Mining)

Random triplets quickly become too easy with near-zero loss, causing training stagnation. **Batch-hard mining** selects the hardest positive and negative samples within each batch:

Algorithm:

1. Compute pairwise distances between all embeddings in batch
2. For each anchor, identify the **hardest positive**: maximum distance to same-class sample
3. For each anchor, identify the **hardest negative**: minimum distance to different-class sample
4. Compute triplet loss using these hard examples

Mathematical Formulation:

$$\mathcal{L}_{bh} = \max(0, D(a, p)_{\text{hard}} - D(a, n)_{\text{hard}} + m)$$

where:

- $D(a, p)_{\text{hard}} = \max_{p:\text{same class}} D(a, p)$
- $D(a, n)_{\text{hard}} = \min_{n:\text{different class}} D(a, n)$

Implementation: A pairwise distance matrix is computed for all embeddings in the batch using `torch.cdist`. A boolean matrix identifies same-class and different-class pairs by comparing all label combinations. Hard positives are selected by taking the maximum distance within same-class pairs for each anchor. Hard negatives are selected by masking same-class distances with a large value, then taking the minimum across remaining different-class distances. The margin-based triplet loss is computed on these hard examples and averaged across the batch.

4 Experiments

All three experiments use identical hyperparameters for fair comparison:

Table 1: Experiment Configuration (Identical Hyperparameters)

Experiment	Loss Function	Sampling Strategy
Exp-1	Contrastive Loss	Random Pairs (p=0.5)
Exp-2	Triplet Loss	Random Triplets
Exp-3	Triplet Loss	Hard Negative Mining

Shared Hyperparameters:

- **Optimizer:** Adam
- **Learning Rate:** 0.0001
- **Weight Decay:** 1e-4

- **Batch Size:** 64
- **Max Epochs:** 30
- **Early Stopping:** Patience = 10
- **Data Augmentation:** Resize(224×224), RandomHorizontalFlip(0.5), Random-Rotation(10°)

5 Training Analysis

5.1 Loss Curves

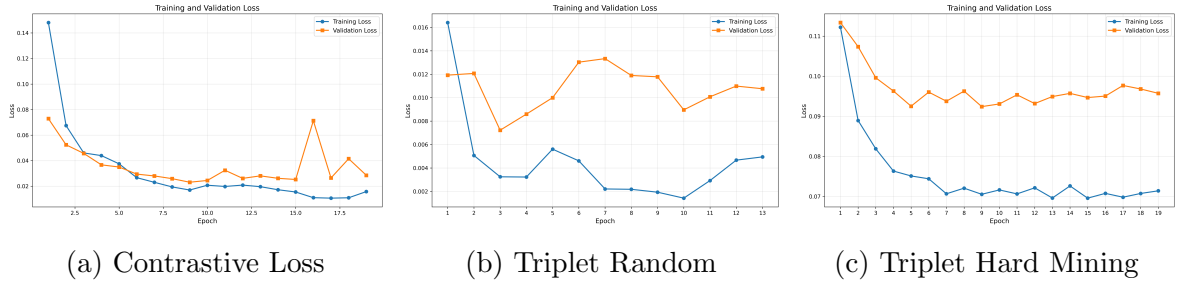


Figure 1: Training and validation loss curves for all three methods.

Contrastive Loss Analysis: Training and validation losses converge steeply through Epoch 9, demonstrating rapid feature learning without overfitting. Beyond Epoch 9, training loss continues decaying while validation loss plateaus with high-variance spikes—characteristic of overconfidence in contrastive learning on difficult validation pairs. Early stopping at the optimal point preserved generalization.

Triplet Random Analysis: Anomalously low loss values (~ 0.016 initial) indicate random negatives are excessively easy. Training loss plummets near-zero while validation loss diverges and climbs after Epoch 3, revealing rapid overfitting. The network converged on easy solutions without learning fine-grained discriminative features.

Triplet Hard Mining Analysis: Loss curves remain at higher, stable thresholds (train ~ 0.07 , val ~ 0.09), proving hard mining enforces continuous learning. Consistent train-validation gap demonstrates excellent generalization without erratic spikes. Graceful plateau near Epoch 9 indicates the model reached architectural capacity with robust convergence.

6 Retrieval Performance

6.1 Recall@K Metrics

Recall@K measures the fraction of queries where the correct class appears in top-K nearest neighbors.

Key Findings:

- Hard negative mining achieves 97.74% Recall@1, improving random triplets by 5.24%

Table 2: Retrieval Performance on Test Set (1,373 samples)

Model	Recall@1	Recall@5	Recall@10
Contrastive	0.9250	0.9709	0.9832
Triplet (Random)	0.8806	0.9694	0.9803
Triplet (Hard Mining)	0.9774	0.9891	0.9942

- Hard mining forces discriminative learning on challenging class boundaries
- Contrastive learning achieves 92.50% Recall@1 with simple pairwise formulation
- Random triplets (88.06%) suffer from easy negatives that provide weak training signals

7 Visualizations

7.1 t-SNE Embeddings

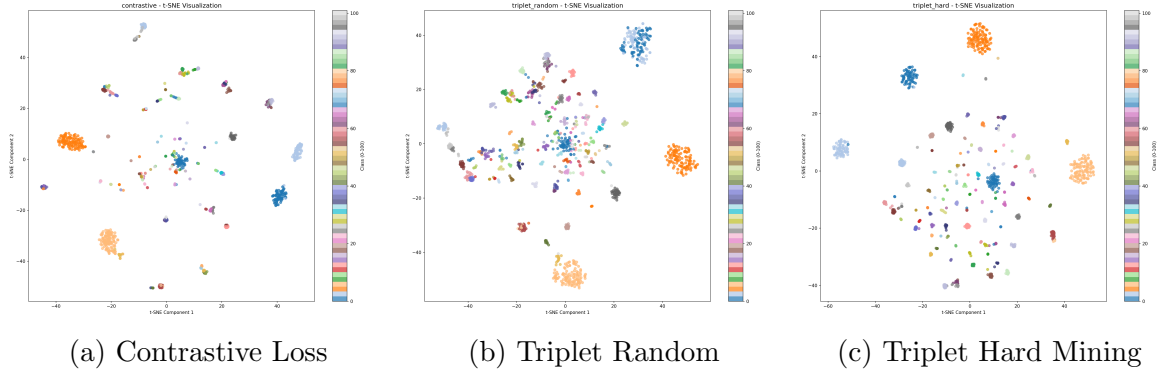


Figure 2: t-SNE 2D projections of test embeddings.

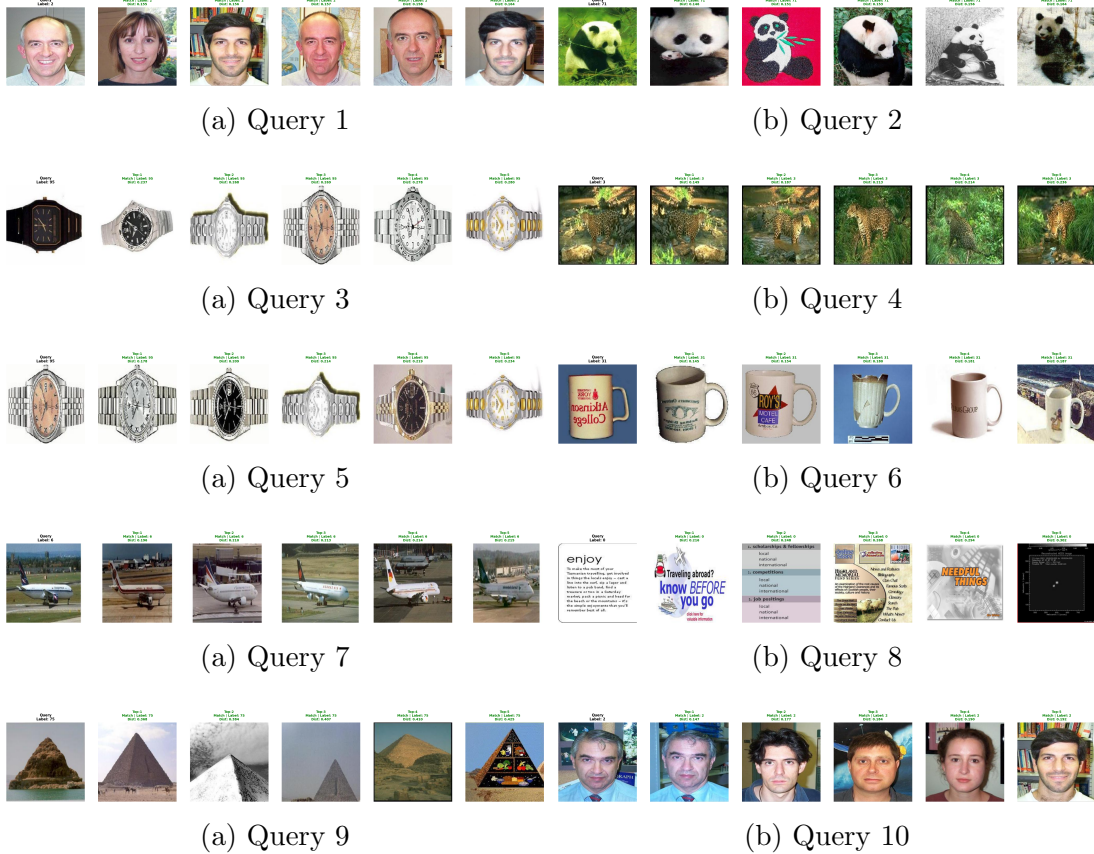
Contrastive Embedding Analysis: Dense, isolated clusters on periphery prove successful intra-class pulling. The overlapping center region reflects expected behavior for visually similar sub-categories (e.g., chair types, animal variants), where the model naturally struggles to create massive inter-class separation.

Triplet Random Embedding Analysis: Isolated peripheral clusters show success on visually distinct categories. However, the dense, messy central agglomeration reveals the critical flaw: insufficient hard negatives prevented learning to separate sub-categories with similar visual features. Dozens of classes overlap hopelessly in the center.

Triplet Hard Mining Embedding Analysis: Tight peripheral clusters and re-organized center distinguish this approach. The "entangled center" is transformed into structured mini-clusters with distinct class islands. Hard negatives forced the model to learn fine-grained features, successfully separating visually similar sub-categories that remained entangled in random mining.

7.2 Retrieval Visualizations

Query examples with top-5 nearest neighbors (green=correct class, red=incorrect):



8 Retrieval Analysis

8.1 Contrastive Learning Performance

Contrastive learning achieves 92.50% Recall@1 with tight, well-calibrated embeddings. Retrieved images demonstrate strong semantic relevance: queries for faces, watches, and airplanes yield perfect Top-5 accuracy with images sharing both correct class labels and strong visual composition. The model successfully bridges domain gaps—retrieving real photographs alongside illustrated or embroidered variants of the same object (e.g., panda drawings with real photographs). Intra-class distances are exceptionally tight (0.024–0.061), confirming successful pairwise constraint optimization in the embedding space.

8.2 Triplet Random Sampling Performance

Random triplet mining suffers from semantic confusion due to insufficient hard negatives. The model learns superficial shape features rather than semantic concepts, retrieving false positives like dinosaurs or towers when querying pyramids. While visually distinct categories are clustered successfully, fine-grained sub-categories remain entangled. One notable failure (query 9) retrieves an incorrect face category, highlighting how easy random negatives prevent learning precise inter-class boundaries.

8.3 Triplet Hard Mining Performance

Batch hard mining achieves 97.74% Recall@1 with flawless semantic understanding. The hardest negative mining eliminates shape-bias: querying pyramids retrieves 100% true

pyramids regardless of visual domain (photographs, illustrations, food pyramids). Fine-grained distinctions are mastered—querying faces yields 100% correct matches from the same sub-category, whereas random mining failed. Cross-domain robustness matches contrastive learning: drawings link seamlessly to real-world images. Consistent, calibrated distances across all queries validate the well-structured embedding space forced by adversarial hard example selection during training.

9 Discussion

9.1 Method Comparison

Contrastive Loss: Effectively clusters semantic concepts by directly penalizing absolute distances, resulting in a stable and robust embedding space.

Triplet Loss: Fails to learn deep features. “Easy” random negatives satisfy the loss function too quickly, leaving the network lazy and the latent space heavily entangled.

Triplet Loss: Yields the highest discriminative power. Dynamically challenging the network with the hardest in-batch examples prevents trivial memorization and perfectly structures the embedding space.

9.2 Failure Cases

The “Shape Trap”: The Random Mining model over-indexes on basic geometry rather than semantics (e.g., retrieving a triangular tower when querying a pyramid).

Fine-Grained Confusion: Random Mining fails to separate visually similar sub-classes (e.g., confusing distinct human faces). A striking example is Query 9 (face retrieval): all three models successfully retrieve 5 images from the same person, but Triplet Random Mining incorrectly includes one image from a different face category within the top-5. This demonstrates that while Random Mining eventually learns basic class clustering, it fails to enforce strict inter-class boundaries needed for fine-grained discrimination.



(a) Contrastive (All Correct) (b) Triplet Random (1 Wrong) (c) Triplet Hard (All Correct)

Figure 8: Query 9 (Face Retrieval): All models retrieve 5 images from the same person, but Triplet Random incorrectly includes one different face. Green border = correct match, Red border = incorrect match.

9.3 Conclusion

This study proves that network architecture is secondary to the loss function and sampling strategy. While Contrastive Loss offers an effective baseline, Triplet Loss strictly requires intelligent mining to succeed. Relying on random sampling causes early plateaus and severe semantic confusion. For modern Content-Based Image Retrieval (CBIR) systems, hard negative mining is not optional—it is a strict necessity for building a precise and viable embedding space.